

GeoCLIP Reproducibility Tutorial

Objective

The objective of this tutorial is to document the workflow used to generate geospatial embeddings with GeoCLIP and to analyze the resulting representations.

Environment Setup

A separate Conda environment was created for GeoCLIP to avoid dependency conflicts with the SatCLIP environment.

The final working environment included:

- Python 3.11
- PyTorch
- GeoCLIP
- Transformers 4.37.2
- NumPy 1.26.4
- pandas
- scikit-learn
- matplotlib

The main compatibility issue was related to the `transformers` package. A newer version required a more recent PyTorch release, so `transformers==4.37.2` was used instead.

Coordinate-Based Embeddings

GeoCLIP can generate embeddings directly from geographic coordinates using its location encoder.

For this experiment, the 32 Mexican state capitals were used. Each city was represented by its latitude and longitude.

The workflow was:

1. Define the list of Mexican state capitals.
2. Store latitude and longitude for each capital.
3. Pass the coordinates to GeoCLIP's location encoder.
4. Generate a 512-dimensional embedding for each city.
5. Save the resulting embeddings as a CSV file.
6. Compute cosine similarities between city embeddings.
7. Apply PCA to visualize the latent space.

The coordinate-based embedding process can be summarized as:

$$(\text{latitude}, \text{longitude}) \rightarrow \text{GeoCLIP Location Encoder} \rightarrow z$$

where:

$$z \in \mathbb{R}^{512}$$

Image-Based Embeddings

GeoCLIP also includes an image encoder based on CLIP. This makes it possible to generate embeddings directly from images.

For this experiment, representative images from selected Mexican cities were collected and stored locally. Each image was processed using GeoCLIP’s visual encoder.

The workflow was:

1. Store city images in `models/geoclip/images/mexico_cities/`.
2. Load each image.
3. Preprocess the image using the CLIP processor.
4. Pass the processed image to GeoCLIP’s image encoder.
5. Generate a 512-dimensional visual embedding.
6. Save all image embeddings as a CSV file.
7. Apply PCA and cosine similarity analysis.

The image-based embedding process can be summarized as:

$$\text{image} \rightarrow \text{GeoCLIP Image Encoder} \rightarrow z$$

where:

$$z \in \mathbb{R}^{512}$$

Generated Files

The main scripts developed for the GeoCLIP experiments were:

- `models/geoclip/run_mexico_capitals.py`
- `models/geoclip/run_mexico_image_geolocation.py`
- `models/geoclip/extract_all_image_embeddings.py`
- `models/geoclip/pca_image_embeddings.py`

The main outputs were:

- `geoclip_mexico_capitals_embeddings.csv`
- `geoclip_mexico_capitals_similarity.csv`
- `geoclip_mexico_capitals_pca.csv`
- `geoclip_image_embeddings.csv`
- `geoclip_image_embeddings_similarity.csv`
- `geoclip_image_embeddings_pca.csv`

Key Takeaway

GeoCLIP supports two complementary embedding-generation modes. The first uses geographic coordinates through the location encoder, while the second uses images through the visual encoder.

This makes GeoCLIP useful for comparing coordinate-based and image-based geospatial representations within the same latent embedding framework.